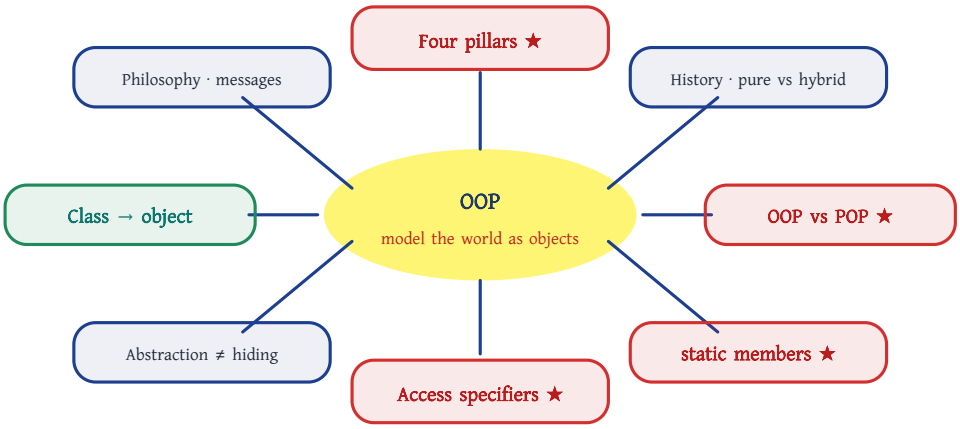
Object-Oriented Programming

Philosophy · Pillars · OOP vs POP · Classes · Access · Static: exam-ready in 12 sheets



↑ the whole lesson on one page: red = high-yield, green = the idea everything else hangs on

Syllabus coverage

The OOP syllabus opens with "**Philosophy of OOP; Advantages of OOP over structured programming; Encapsulation, classes and objects, access specifiers, static and non-static members**". Constructors, inheritance in depth, polymorphism and templates have the next two lessons.

High-yield list

① Object = identity + state + behaviour
② Class = blueprint; object = instance
③ Four pillars = A PIE
④ OOP is bottom-up; POP is top-down
⑤ Encapsulation = bundle; hiding = private
⑥ Java protected = pkg + subclasses
⑦ C++ class → private, struct → public
⑧ static = one copy per class
⑨ static method: no `this`
⑩ Simula 67 = first OO language

Legend: **yellow** = key term · **red circle** = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

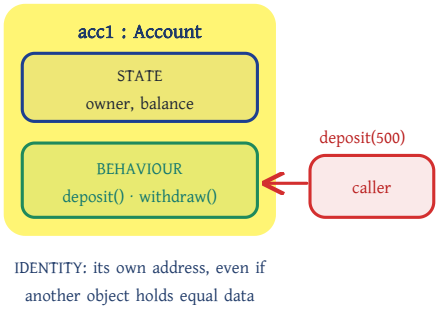
Contents

✓ Philosophy: objects & messages … p. 2
✓ The four pillars … p. 3
✓ Abstraction vs encapsulation … p. 4
✓ Advantages: OOP vs structured … p. 5
✓ History; pure vs hybrid languages … p. 6
✓ Classes and objects … p. 7
✓ Access specifiers: Java & C++ … p. 8
✓ Encapsulation in code: getters/setters … p. 9
✓ Static vs non-static members … p. 10
✓ Night-before sheet … p. 11
✓ Practice MCQs + written … p. 12

1 The Philosophy of OOP: Objects and Messages

Definition: **Object-oriented programming** is a paradigm that builds a program out of **objects** — units that hold their own data and the operations on that data — which cooperate by **sending messages** to one another. Its philosophy in one line: *"program the way we think about the real world."* A car, a student or a bank account is a thing that **has** something (state) and **does** something (behaviour); OOP writes it the same way.

> data + code travel together



three parts of every object

An object has three things (Booch): **Identity** — what makes it *this* object: two accounts with the same owner and balance are still two objects (different addresses / references).
State — the current values of its attributes (fields, data members).
Behaviour — what it can do: its methods (member functions), which read or change the state.
Message passing — `acc1.deposit(500)` sends the message "deposit 500" to the receiver *acc1*: receiver + method name + arguments. The receiver decides how to respond.

Real-world thing	State (has)	Behaviour (does)	In code
Car	colour, engine, speed	start, stop, accelerate	fields + methods of *Car*
Student	name, id, grades	study, sitExam	fields + methods of *Student*
Bank account	owner, balance	deposit, withdraw	fields + methods of *Account*

Memory trick: "**Nouns become classes, adjectives become fields, verbs become methods.**" Read a problem statement aloud and you have a first class design.

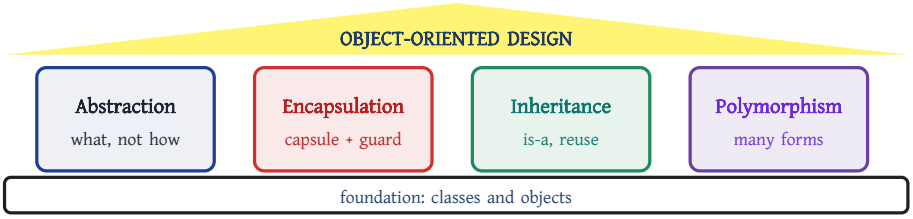
Exam trap: **Identity is not state.** Two objects with equal fields are *equal* but not *identical*: in Java `a == b` compares identity, `a.equals(b)` compares state.

Worked example ✍ spot the parts:
"A library lends books to members." Classes: **Library, Book, Member**. Book state: title, isbn, onLoan. Message: `lib.lend(book, member)` → receiver **lib**.

Where it came from: Alan Kay, who coined the term, described OOP as "messaging, local retention and hiding of state, and late binding". Those three ideas reappear below as **message passing**, **encapsulation** and **polymorphism**.

2 The Four Pillars of OOP

Definition: Four principles make a language object-oriented: **abstraction** (show *what*, hide *how*), **encapsulation** (bundle data with the code that uses it and control access), **inheritance** (a new class acquires the members of an existing one) and **polymorphism** ("one interface, many forms"). Classes and objects are the **building blocks**; the pillars are the **principles** built on them.



A PIE: four pillars on one floor of classes and objects

Pillar	One-line meaning	Tool in Java / C++	Everyday picture
Abstraction	expose essential features, hide implementation	abstract class, interface, public methods	steering + brake, engine hidden
Encapsulation	data + methods in one unit; access controlled	class + `private` fields + methods	medicine capsule
Inheritance	child class reuses and extends a parent	`extends` (Java), `: public` (C++)	Car is-a Vehicle
Polymorphism	same call, different behaviour by type	overloading (compile time), overriding (run time)	*draw()* on circle vs square

Inheritance at pillar level: A class is **derived** from a **base** class and gets its members: **is-a** relationship (Dog is-a Animal), unlike **has-a** = composition (Car has-a Engine). Types and constructor order: next lesson. Java: a class `extends` only **one** class but may `implements` many interfaces.

Polymorphism at pillar level: **Compile-time** (static binding): function / operator **overloading** — chosen by the argument list. **Run-time** (dynamic binding): **overriding** through a base-class reference or pointer; in C++ the function must be `virtual`. Details: a later lesson.

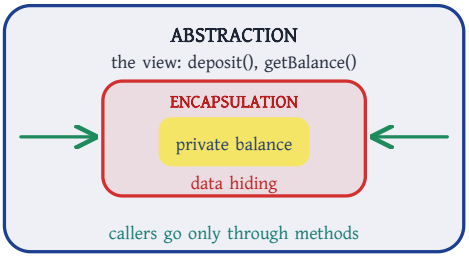
Memory trick: **A PIE** = **A**bstraction, **P**olymorphism, **I**nheritance, **E**ncapsulation. "Which is *not* a pillar?" — overloading, class, object, compilation.

Exam trap: Some books list only **three** pillars (no abstraction). Choose four unless the options force three. **Class** and **object** are not pillars.

Worked example ✍ name the pillar:
`private int pin;` → **encapsulation**. `Savings extends Account` → inheritance. Two `add()` with different parameters → polymorphism.

3 Abstraction vs Encapsulation vs Data Hiding

Definition: Three words, three different jobs. **Abstraction** is a **design** idea: decide which features the outside world needs and ignore the rest. **Encapsulation** is an **implementation** mechanism: wrap the data and the methods that use it into one class. **Data hiding** is the **protection** step inside encapsulation: make the data `private` so only the class's own methods can touch it.



outside → interface → capsule → hidden data

three layers of one class

Point	Abstraction	Encapsulation
Question	**what** does it do?	**how** is it packed and guarded?
Level	design	implementation
Hides	complexity	data (by access control)
Tool	abstract class, interface	class + private + getters/setters
Example	ATM shows "withdraw"	*balance* private in *Account*

Abstract class vs interface (pillar level)

Point	Abstract class	Interface (Java)
Object creation	cannot be instantiated	cannot be instantiated
Methods	abstract **and** concrete	abstract; `default/static` bodies since Java 8
Fields	any kind, any access	only `public static final` constants
Constructor	yes (run by subclasses)	no
A class may use	**one** (`extends`)	**many** (`implements`)
In C++	≥ 1 pure virtual function `= 0`	no keyword: a class of only pure virtuals

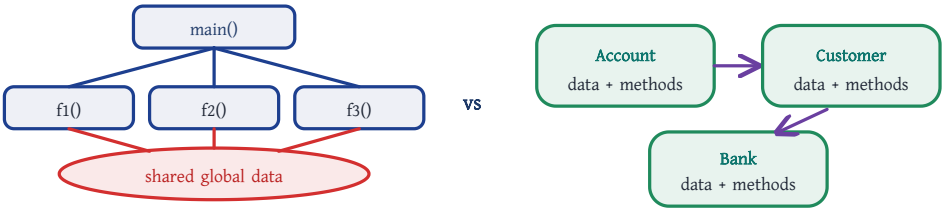
Memory trick: "**Abstraction is the menu, encapsulation is the kitchen door, data hiding is the lock on it.**" The customer orders from the menu and never walks into the kitchen.

Exam trap: **Encapsulation ≠ data hiding.** A class with all-public fields is still encapsulated (bundled) but hides nothing. And `private` is a **compile-time** check, not security: Java reflection can still read it.

Worked example ✍ classify: "Shows only essential features" → **abstraction**. "Wraps data and functions into a single unit" → **encapsulation**. "Prevents direct access to fields" → data hiding.

4 Advantages of OOP over Structured Programming

Definition: **Structured (procedural) programming** — C, Pascal, FORTRAN — splits a problem **top-down** into functions, and data moves freely between them, often as globals. **OOP** builds **bottom-up** from classes, keeping each piece of data inside the object that owns it. The advantages all follow from that one change: **data stays with its code.**



left: top-down functions over shared data · right: objects exchanging messages

Aspect	Structured / procedural (POP)	Object-oriented (OOP)
Design approach	**top-down**	**bottom-up**
Unit of program	function / procedure	class and object
Emphasis	algorithm (what to do)	data (what exists)
Data and functions	separate; global data shared	bundled (encapsulation)
Data security	low: no access specifiers	high: private / protected / public
Reuse	functions and libraries	also inheritance and composition
Adding a feature	often edit existing functions	add a subclass; old code untouched
Best for	small programs, drivers, firmware	large, long-lived systems
Examples	C, Pascal, FORTRAN, COBOL	C++, Java, C#, Python, Smalltalk

Seven advantages to write: ① **data hiding** · ② **reusability** (inheritance, composition) · ③ **modularity** — each class debugged alone · ④ **easy maintenance** · ⑤ **extensibility** — new subclass, old code untouched · ⑥ **flexibility** (polymorphism) · ⑦ **real-world modelling**, **scalability**.

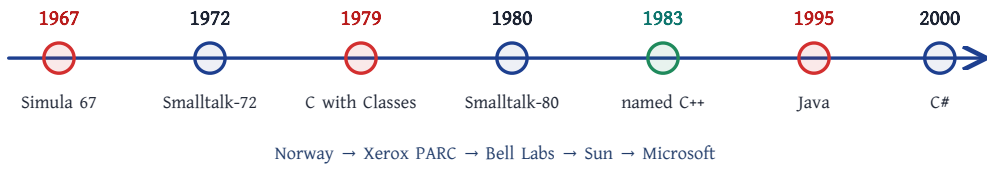
Exam trap: POP is **not** "no reuse": functions and libraries are reuse. OOP adds reuse *with extension*. OOP also has costs: larger programs, extra indirection, a steeper design effort — not best for a 50-line script. Good design = **low coupling, high cohesion**.

Memory trick: **"Top-down cuts the task; bottom-up builds the things."** Coupling = how much classes lean on each other (keep it low); cohesion = how focused one class is (keep it high).

Worked example ✍ the cost of global data: A C payroll keeps *salary[]* global; 30 functions touch it. One salary turns negative: all **30 functions** are suspects. In OOP only *Employee.setSalary()* can change it, and it rejects negatives.

5 History of OOP; Pure vs Hybrid Languages

Definition: OOP began in Norway with **Simula 67** (Ole-Johan Dahl and Kristen Nygaard), the first language with classes, objects, inheritance and virtual procedures. **Smalltalk** at Xerox PARC (Alan Kay) made "everything is an object" the rule. **C++** (Bjarne Stroustrup, Bell Labs) and **Java** (James Gosling, Sun) took the idea mainstream.



five dates carry most of the MCQs

Language	Year	Creator / place	Remember it for
Simula 67	1967	Dahl & Nygaard, Norwegian Computing Center	**first OO language**
Smalltalk	1972 (public: -80)	Alan Kay, Dan Ingalls, Adele Goldberg; Xerox PARC	pure OO; Kay coined "OOP"
C++	1979 → 1983 → 1985	Bjarne Stroustrup, Bell Labs	"C with Classes" renamed C++ in 1983; released 1985
Eiffel	1986	Bertrand Meyer	design by contract
Java	1995	James Gosling, Sun (began 1991 as *Oak*)	"write once, run anywhere", JVM
C#	2000	Anders Hejlsberg, Microsoft	.NET

Category	Meaning	Examples
Pure OO	everything, even numbers and classes, is an object	Smalltalk, Ruby, Eiffel
Hybrid / partial OO	OO plus procedural features; primitives are not objects	C++, Java, Objective-C, Python (multi-paradigm)
Object-based	objects and encapsulation, **no inheritance**	Ada 83, Visual Basic 6

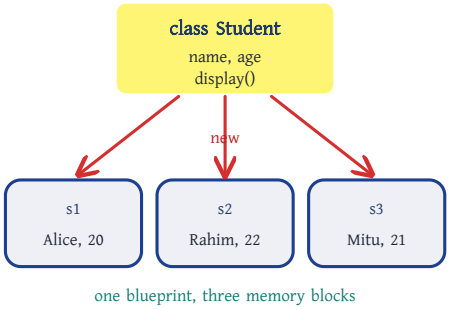
Memory trick: "**67 · 72 · 79/83 · 95**": Simula, Smalltalk, C++ (born, named), Java. Stroustrup's language was "C with Classes" for four years.

Exam trap: **Java is not pure OO**: `int`, `char` etc. are primitives, and `static` members live outside objects. **C++ is hybrid**: a valid C++ program needs no class at all.

Worked example ✍ clue chain:
"First language to support classes and inheritance, designed for simulation" → **Simula 67**. "Bell Labs, 1983" → C++.

6 Classes and Objects

Definition: A **class** is a user-defined type: a **blueprint** that declares data members (attributes) and member functions (methods). An **object** is an **instance** of a class — a region of memory holding one set of those attributes. The class is a **logical** entity; the object is a **physical** one. Memory for instance data is allocated **per object**, not when the class is declared.



one blueprint, three memory blocks

dashed = no memory for data

```
class Student {
    String name;        // state
    int age;            // state
    void display() {    // behaviour
        System.out.println("Name: " + name
                           + ", Age: " + age);
    }
}
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(); // object
        s1.name = "Alice";
        s1.age = 20;
        s1.display(); // Name: Alice, Age: 20
    }
}
```

Point	Class	Object
What it is	blueprint, user-defined type	instance of that type
Entity	logical	physical (occupies memory)
How many	declared **once**	created **many** times
Created by	`class` keyword	`new` (Java); also `Student s;` (C++)
Field memory	none (static members excepted)	one copy of every non-static field

Memory trick: "**Class is the house plan; objects are the houses.**" You cannot live in a plan, and every house has its own furniture (fields) but the same rooms (methods).

Exam trap: In Java `Student s1;` creates only a **reference** (null), **no object**; `new` makes the object. In C++ `Student s1;` **does** create an object. C++ `sizeof` an empty class = **1**, never 0.

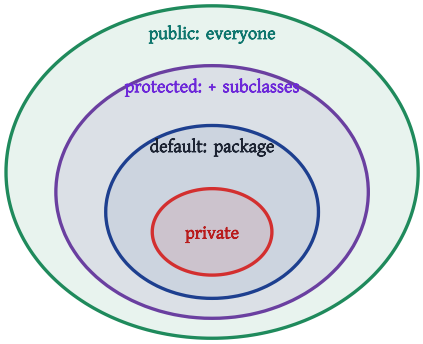
Worked example ✍ count objects:

`a = new Student(); b = a;` plus `Student c;` → objects: **1**; references: 3.

Same class in C++: `class Student { public: string name; int age; void display(); };` — without `public:` every member is private. Methods are stored **once**, shared by all objects.

7 Access Specifiers: Java and C++

Definition: **Access specifiers** (Java: *access modifiers*) set who may use a member. They are how a class enforces **data hiding**. Java has **four** levels — `private`, default (no keyword, "package-private"), `protected`, `public`; C++ has **three** keywords — `private`, `protected`, `public`, plus `friend` to grant exceptions.



Java: each ring adds callers

Java modifier	Same class	Same package	Subclass, other pkg	Other pkg
`private`	✓	✗	✗	✗
default (none)	✓	✓	✗	✗
`protected`	✓	✓	✓*	✗
`public`	✓	✓	✓	✓

* only **through inheritance** — via the subclass's own reference (`this` or a subclass object), not via a plain parent object.

C++ specifier	Own class + friends	Derived class	Outside
`private`	✓	✗	✗
`protected`	✓	✓	✗
`public`	✓	✓	✓

Default when you write nothing	Members	Inheritance mode
C++ `class`	**private**	private (`class D : B`)
C++ `struct`	**public**	public (`struct D : B`)
Java member	package-private (default)	— (always `extends`)
Java top-level class	only `public` or default allowed	one `public` class per .java file

Memory trick: Java, widest last: "**Private Dogs Protect People**" = private → default → protected → public. C++: "**class keeps secrets, struct tells all.**"

Exam trap: **Java default ≠ C++ default.** Java default = package; C++ class default = private. Java `protected` is **wider** than default (adds subclasses elsewhere). `static`, `final`, `friend` are **not** access specifiers.

Worked example ✍️ who can read x?:
`package a; public class P { protected int x; }`
Class *Q* in *a*: **yes**. `R extends P` in package *b*: yes, as `this.x`; via `new P().x`: no. *S* in *b*, not a subclass: no.

8 Encapsulation in Code: Getters, Setters, Validation

Definition: **Encapsulation** = bundling data and the methods that operate on it into one class, and restricting direct access to the data. The standard recipe: make every field `private`, expose `public` **getters** (accessors) to read and **setters** or operation methods (mutators) to change, and put the **validation** inside them so the object can never hold an invalid state.

```
class BankAccount {
  private String owner;
  private double balance;  // hidden
  public BankAccount(String o, double b) {
    owner = o;  balance = b > 0 ? b : 0;
  }
  public void deposit(double amt) {
    if (amt > 0) balance += amt;
  }
  public boolean withdraw(double amt) {
    if (amt <= 0 || amt > balance)
      return false;      // refused
    balance -= amt;  return true;
  }
  public double getBalance() { return balance; }
  public String getOwner()   { return owner; }
  public void setOwner(String o) {
    if (o != null && !o.isEmpty()) owner = o;
  }
}
```

Worked example ✍️ trace the balance:
`BankAccount a = new BankAccount("Rahim", 1000);`
`a.deposit(500);` → 1500
`a.withdraw(2000);` → **false**, still 1500
`a.withdraw(300);` → 1200
`a.deposit(-50);` → ignored, 1200
`a.balance = 9999;` → **compile error**: balance has private access
Final `a.getBalance()` = **1200.0**

What encapsulation buys: **Integrity** — no negative balance can ever exist. **Read-only fields** — a getter with no setter. **Freedom to change** — store balance in paisa as `long` tomorrow; callers of `getBalance()` never notice. **One place to debug** — every change passes through three methods.

Member	Access	Kind	Why
`balance`, `owner`	`private`	field	data hiding
`getBalance()`, `getOwner()`	`public`	getter / accessor	read without exposing the field
`deposit()`, `withdraw()`, `setOwner()`	`public`	mutator	change only after a check

Memory trick: "**Fields private, methods public, checks inside.**" Three words for every encapsulation program you will ever write.

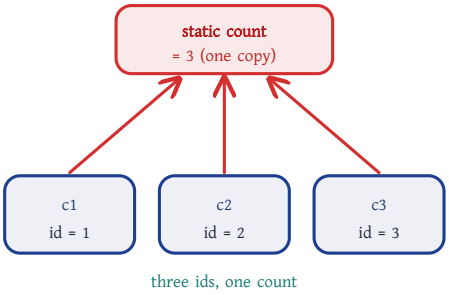
Exam trap: A public setter with **no check** is encapsulation in name only. `private` is per **class**, not per object: one Account method may read another Account's balance.

C++ version: Same design: `class BankAccount { double balance; public: … };` — *balance* is private **by default**; the getter is marked `const`.

9 Static vs Non-Static Members

Definition: A **static** member belongs to the **class**: one copy, shared by every object, existing even when no object exists, accessed as `ClassName.member`. A **non-static** (instance) member belongs to each **object**: every object has its own copy. A **static method** has no `this`, so it **cannot touch instance members** directly.

```
class Counter {
    static int count = 0;  // one, shared
    int id;                // one per object
    Counter() {
        count++;
        id = count;
    }
}
// in main():
Counter c1 = new Counter();
Counter c2 = new Counter();
Counter c3 = new Counter();
System.out.print(c1.id + " " + c3.id
                 + " " + Counter.count);
// Output: 1 3 3
```



all arrows reach the same box

Step	count	c1.id	c2.id	c3.id
class loaded (static init)	0	–	–	–
new Counter() → c1	1	1	–	–
new Counter() → c2	2	1	2	–
new Counter() → c3	**3**	1	2	3

Point	Static	Non-static (instance)
Belongs to	the class	each object
Copies	one	one per object
Created	once, when the class is loaded / program starts	with each `new`
Method can use	only static members (no `this`, `super`)	static **and** instance members
C++ `sizeof`	**not counted** in the object	counted

Memory trick: "**Static = the class's notice board; instance = each student's notebook.**" Why is `main` static? The JVM calls it before any object exists.

Exam trap: Inside `static void f()`, `id++` or `this.id` is a **compile error**; `new Counter().id` is fine. C++ needs one definition outside: `int Counter::count = 0;` Static blocks run **once**, before any constructor.

Worked example ✍️ sizeof: `class A { int x; static int y; void f(); };` with 4-byte int: `sizeof(A)` = **4** — static *y* and function *f* are not in the object. Empty class: 1.

Night-Before-the-Exam Sheet

Fact sheet:

Object = identity + state + behaviour

Class = blueprint; object = instance

Pillars: abstraction, encapsulation, inheritance, polymorphism

Message = receiver + method + arguments

POP top-down; OOP bottom-up

Low coupling, high cohesion

Simula 67 = first OO language (Norway)

Smalltalk = pure OO; Alan Kay

C++: Stroustrup, 1979 → named 1983

Java: Gosling, Sun, 1995 (Oak)

Java: private · default · protected · public

C++: class → private, struct → public

static = one copy per class

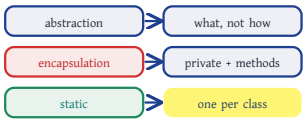
static method: no `this`, no instance access

Top 12 one-liners for MCQs

1. OOP philosophy: model the real world as objects
2. Encapsulation = data + functions in one unit
3. Abstraction = show essential, hide implementation
4. Data hiding = making fields private
5. Inheritance = is-a; composition = has-a
6. Overloading = compile time; overriding = run time
7. Java `protected` = package + subclasses
8. Java default = package-private
9. Java: one class `extends`, many `implements`
10. Object memory per instance, not per class
11. C++ empty class: `sizeof` = 1
12. Java is not pure OO (primitives)

Top traps: encapsulation ≠ data hiding · abstraction is design, encapsulation is implementation · `static` is not an access specifier · Java default ≠ C++ default · `Student s;` in Java makes no object · static members not in `sizeof` · POP also reuses code.

All tricks together: nouns → classes, verbs → methods · A PIE · menu, kitchen door, lock · top-down cuts the task, bottom-up builds the things · 67 · 72 · 79/83 · 95 · house plan vs houses · Private Dogs Protect People · notice board vs notebook.



three ideas, three lines

Worked example ✍️ one keyword changes the output: In the *Counter* class, drop `static` from *count*: each object now starts its own count at 0, so every *id* is 1 and `c1.count` is 1 — output "`1 1 1`" instead of "1 3 3" (and `Counter.count` no longer compiles).

✓ Practice MCQs (exam pattern, 10 marks)

1. Wrapping data and the functions that operate on it into a single unit is:
(a) Inheritance (b) Polymorphism
(c) Encapsulation (d) Abstraction

2. An object is:
(a) an instance of a class (b) a function returning a class
(c) a keyword (d) a header file

3. Which was the first object-oriented language?
(a) C++ (b) Smalltalk
(c) Java (d) Simula 67

4. In Java, a member with no access modifier is visible to:
(a) its class only (b) its package
(c) all subclasses everywhere (d) every class

5. In C++, members of a `class` are by default:
(a) public (b) protected
(c) private (d) static

6. Which concept shows only essential features and hides implementation?
(a) Abstraction (b) Inheritance
(c) Overloading (d) Polymorphism

7. A static method in Java cannot:
(a) be called by class name (b) use static fields
(c) be public (d) use `this`

8. OOP follows which design approach?
(a) top-down (b) bottom-up
(c) left-right (d) none

9. Which is not an access specifier?
(a) private (b) public
(c) protected (d) static

10. `class A { int x; static int y; };` With 4-byte int, `sizeof(A)` is:
(a) 8 (b) 4
(c) 0 (d) 1

Answer key → 1-c · 2-a · 3-d · 4-b · 5-c · 6-a · 7-d · 8-b · 9-d · 10-b

Written question: State the philosophy of OOP and its four pillars. Give five advantages of OOP over structured programming, and write a class showing encapsulation with access specifiers and one static member. 10 marks

1. **Philosophy (2)** — model the problem as objects with identity, state and behaviour that cooperate by message passing; "program the way we think about the real world". 2
2. **Pillars (2)** — abstraction (what, not how), encapsulation (bundle + guard), inheritance (is-a reuse), polymorphism (one interface, many forms). 2

3. **Advantages (3)** — data hiding, reusability through inheritance, modularity and easy maintenance, extensibility without editing old code, real-world modelling and scalability; POP is top-down with shared data. 3
4. **Code (3)** — class with `private` balance, `public` deposit/withdraw with validation and `getBalance()`, and `static int count` incremented in the constructor; state that count is shared and balance is per object. 3

Before the exam ✓

- ✓ Can name the four pillars
- ✓ Can separate abstraction, encapsulation, hiding
- ✓ Can write the OOP vs POP table
- ✓ Can fill the Java access grid
- ✓ Can trace a static counter
- ✓ Can date Simula, C++, Java